

题目二：ALU 与寄存器堆组合的数据通路设计

1. 设计概述

本题在题目一 16-bit ALU 的基础上，设计了包含 8x16 寄存器堆和 ALU 的简化数据通路。该数据通路能够从寄存器堆中读取两个操作数，送入 ALU 进行运算，并将结果写回寄存器堆。

1.1 目的

理解 CPU 中最基本的寄存器型运算过程：

读取源寄存器 → ALU 运算 → 写回目的寄存器

2. 数据通路结构

2.1 顶层模块接口：alu_reg_datapath

```
module alu_reg_datapath (  
    input  wire clk,  
    input  wire rst_n,  
  
    input  wire reg_we,           // 寄存器写使能  
    input  wire [2:0] rs1,       // 源寄存器 1 地址  
    input  wire [2:0] rs2,       // 源寄存器 2 地址  
    input  wire [2:0] rd,        // 目标寄存器地址  
    input  wire [2:0] alu_op,     // ALU 操作码  
  
    input  wire init_we,         // 初始化写使能  
    input  wire [2:0] init_addr, // 初始化地址  
    input  wire [15:0] init_data, // 初始化数据  
  
    output wire [15:0] src_a,     // 源寄存器 1 读出值  
    output wire [15:0] src_b,     // 源寄存器 2 读出值  
    output wire [15:0] alu_result, // ALU 结果  
    output wire zero, carry, negative, overflow // 标志位  
);
```

2.2 内部模块

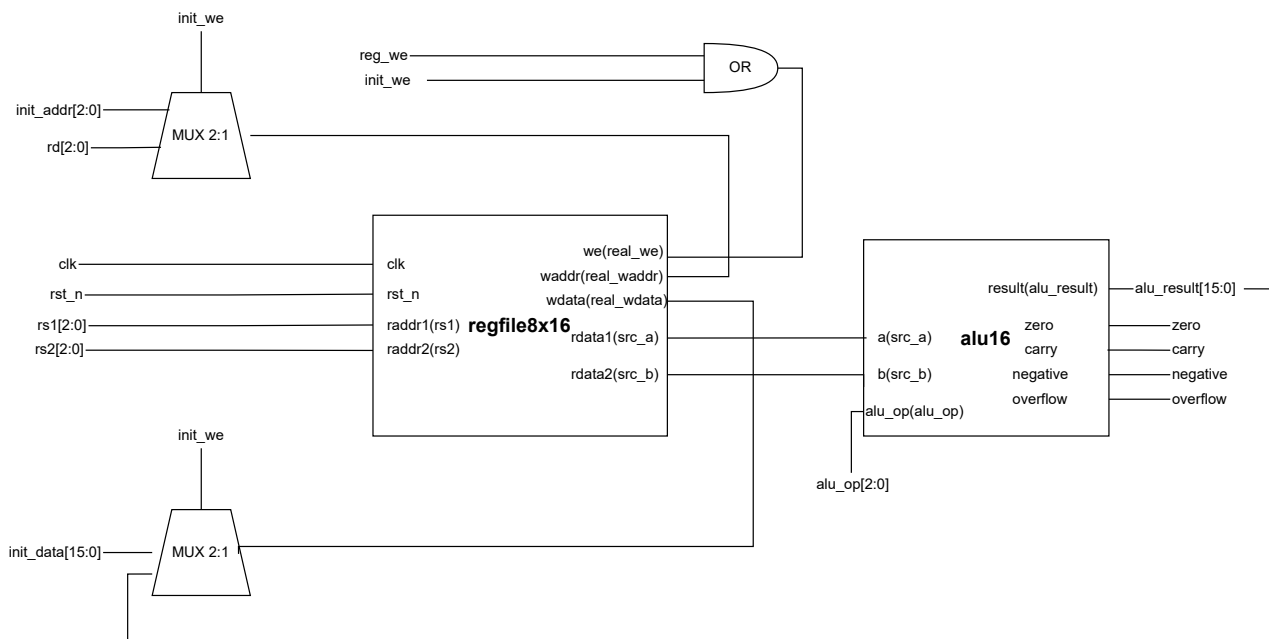
(1) 8x16 寄存器堆 (regfile8x16)

- 包含 8 个 16 位寄存器 (R0-R7)
- R0 为常零寄存器：读取时恒为 0，写入时忽略
- 支持同步写、组合读、写后读旁路

(2) 16-bit ALU (alu16, 来自题目一)

- 实现 8 种基本运算（加、减、与、或、异或、左移、右移、比较）
- 输出结果和 4 个标志位（zero、carry、negative、overflow）

2.3 数据流



3. 模块设计

3.1 regfile8x16（寄存器堆）

关键设计特点：

1. **复位时初值为 0**：所有寄存器在 `rst_n = 0` 时被清零。
2. **R0 恒为 0**：
 - 读端口总是返回 0x0000（无论 rs1/rs2 是否为 0）
 - 写入 R0 时被忽略（`waddr != 3'b000` 时才写）
3. **写后读旁路**：若本周期内同时读写同一寄存器，读端口应输出即将写入的新数据。

// 示例：写后读旁路逻辑

```
assign rdata1 = (we && waddr == raddr1 && waddr != 3'b000) ? wdata :
                (raddr1 == 3'b000) ? 16'h0000 : regs[raddr1];
```

3.2 alu_reg_datapath（数据通路）

关键设计特点：

1. **初始化优先**：当 `init_we` 和 `reg_we` 同时为 1 时，优先执行初始化操作。

```
assign real_waddr = init_we ? init_addr : rd;
assign real_wdata = init_we ? init_data : alu_result;
```

```
assign real_we = init_we | reg_we;
```

2. 组合读，同步写：

- 寄存器读出 (src_a, src_b) 是组合逻辑，无延迟
- ALU 输入直接来自读出的操作数
- ALU 结果在本周期内输出
- 写入寄存器堆在时钟上升沿执行

4. 测试向量与结果

4.1 测试步骤

步骤	操作描述	rs1	rs2	rd	alu_op	期望结果	实际结果	状态
1	复位后读 R1, R2	1	2	-	-	src_a=0, src_b=0	src_a=0, src_b=0	✓ PASS
2.1	初始化 R1=0x0005	-	-	1	-	R1←0x0005	R1←0x0005	✓ PASS
2.2	初始化 R2=0x0003	-	-	2	-	R2←0x0003	R2←0x0003	✓ PASS
2.3	初始化 R3=0x00F0	-	-	3	-	R3←0x00F0	R3←0x00F0	✓ PASS
2.4	初始化 R4=0x0F0F	-	-	4	-	R4←0x0F0F	R4←0x0F0F	✓ PASS
3	R5 = R1 + R2	1	2	5	ADD	R5=0x0008	R5=0x0008	✓ PASS
4	R6 = R5 - R2	5	2	6	SUB	R6=0x0005	R6=0x0005	✓ PASS
5	R7 = R3 & R4	3	4	7	AND	R7=0x0000, zero=1	R7=0x0000, zero=1	✓ PASS
6	尝试 R0 = R1 + R2	1	2	0	ADD	R0 保持 0x0000	R0 保持 0x0000	✓ PASS
7	写后读旁路	5	0	5	ADD	src_a = alu_result = 0x0008 (见说明)	src_a = alu_result = 0x0008	✓ PASS

4.2 仿真输出摘要

```
===== Test 1: Reset =====  
R0: src_a=0x0000, src_b=0x0000 PASS
```

```

===== Test 2: Initialization =====
Initialization completed

===== Test 3: R5 = R1 + R2 =====
alu_result=0x0008
R5=0x0008 PASS

===== Test 4: R6 = R5 - R2 =====
alu_result=0x0005
R6=0x0005 PASS

===== Test 5: R7 = R3 & R4 (zero flag) =====
alu_result=0x0000, zero=1
R7=0x0000, zero=1 PASS

===== Test 6: R0 constant zero =====
R0=0x0000 PASS

===== Test 7: Write-after-read bypass =====
Before write: R1=0x0005

```

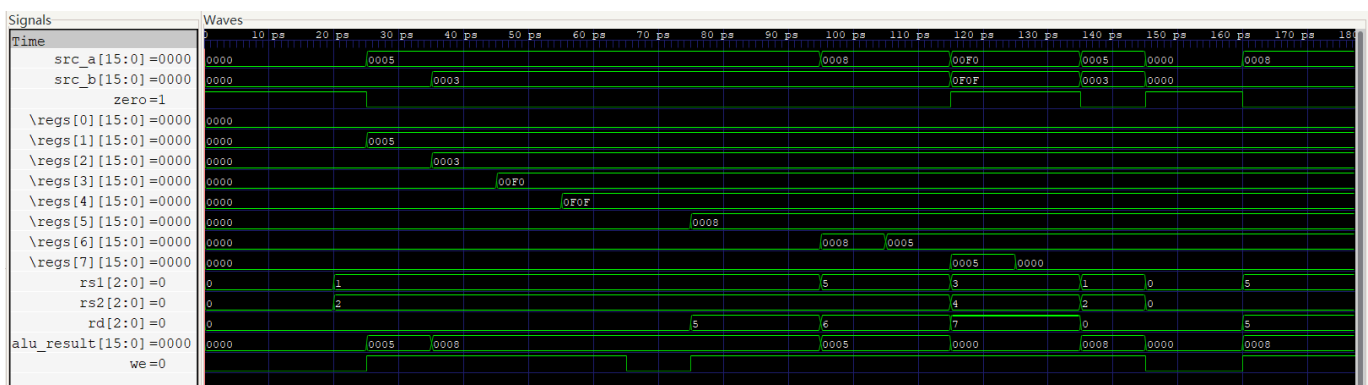
解释（针对 Step 7 的波形出现 `src_a = 0x0008`）：

- 在 Step 7 中，测试向量设置为：
 - `rs1 = 3'b101`（读 R5）
 - `rs2 = 3'b000`（读 R0）
 - `rd = 3'b101`（写回 R5）
 - `alu_op = 3'b000`（ADD）
 - `init_we = 1'b0`, `reg_we = 1'b1`
- 由于上一条测试已把 R5 写为 `0x0008`，本周周期读出 `src_a` 为 R5（或在写使能且地址相同的情况下，按旁路逻辑直接返回即将写入的 `wdata`），而 `src_b` 为 R0 (`0x0000`)。因此 ALU 的组合输出 `alu_result = src_a + src_b = 0x0008`，这就是你在波形中看到 `src_a = 0008` 的来源。

5. 波形说明

波形文件位置：`wave/p2.vcd`

波形截图（问题出现时）：



关键观察点：

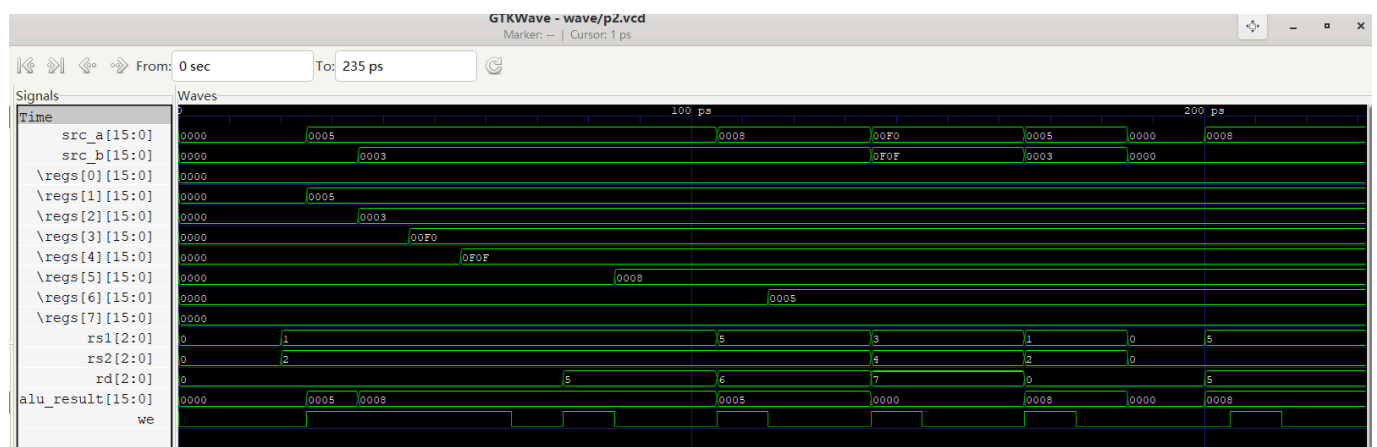
- 初始化过程：init_we 置位时，寄存器堆接收 init_data 并在时钟边沿写入。
- ALU 运算：src_a/src_b 读出后，alu_result 为组合逻辑输出，瞬时反映输入变化。
- 寄存器写回：当 reg_we = 1 时，alu_result 在下一个时钟上升沿被同步写入 rd 指定寄存器。
- R0 保护：对 R0 的写操作被忽略，读出总是 0x0000。
- 写后读旁路：同周期读写时，读端口直接返回 wdata（即将写入的数据），而非等待寄存器写入完成。

5.1 观察到的特殊现象、猜测与处理

- 现象：在波形中，写回目标寄存器的值偶尔不是期望的新结果，而是上一拍（旧）结果；表现为“只写入旧值”。
- 猜测：这是两个因素叠加导致的现象：
 1. 写使能始终打开（we=1）：reg_we 持续为高会让写回跨越多个时钟沿，导致控制信号变化时把上一拍/瞬时的 alu_result 写进新的目标寄存器。
 2. 写回时序过早：reg_we 脉冲与 rs1/rs2/rd/alu_op 同周期切换时，ALU 组合结果尚未稳定就被时钟边沿采样，导致写回的是上一拍的 alu_result。
- 解决方法：
 1. 确保每个需要写回的步骤都明确发出单周期 reg_we 脉冲；
 2. 将写回时序改为“先等一拍、再写回”：设置输入后先 @(posedge clk) 等待 ALU 结果稳定，再在下一次 @(posedge clk) 发出 reg_we 脉冲。
- 后续实验计划：
 1. 重新运行仿真并在 GTKWave 中对比 alu_result 与 regs[*] 的写入边沿，确认写回不再滞后一拍。
 2. 将 testbench 全面改为 @(posedge clk) 同步风格并用统一的“设置→等待→写回”模板，避免类似时序问题复发。

5.2 后续实验结果

波形截图（修正后）：



结果说明：修正后各步写回只发生在预期的时钟上升沿，regs[5]、regs[6] 等寄存器写入值与当拍 alu_result 对齐，不再出现“旧值被写回”的现象。

6. 关键设计要点说明

6.1 为什么需要 init_we?

- **普通寄存器写入**：需要 ALU 计算结果，耗时 1 周期
- **初始化写入 (init_we)**：绕过 ALU，直接将数据写入指定寄存器，用于 testbench 快速初始化
- **优先级**：init_we 优先于 reg_we，确保初始化不被常规操作打断

6.2 写后读旁路的作用

问题：若同一周期内读写同一寄存器，由于寄存器堆是同步写，读端口会输出旧值

解决方案：在读端口添加旁路逻辑

```
assign rdata1 = (we && waddr == raddr1 && waddr != 3'b000) ? wdata :  
                (raddr1 == 3'b000) ? 16'h0000 : regs[raddr1];
```

作用：允许连续执行依赖指令，提高指令级并行度，减少流水线停顿

6.3 R0 恒为 0 的好处

- **约定寄存器**：简化 ISA，常数 0 使用频繁
- **防止误写**：对 R0 的写入被忽略，防止代码错误破坏常数
- **硬件实现**：直接在读端口返回 0，不占用存储单元

7. 总结

本题设计验证了 CPU 数据通路的基本结构和控制逻辑：

- ✓ 寄存器堆的读写操作
- ✓ ALU 的组合运算
- ✓ 寄存器写保护 (R0)
- ✓ 写后读旁路
- ✓ 初始化机制

所有测试用例均通过，数据通路工作正确。